CSE311L: DATABASE SYSTEMS LAB

Database Implementation

Auvryon - Intelligent Digital Finance & Payment Ecosystem

MySQL database: schema, constraints and sample data

Group	1
Course / section	CSE311L.3 (Section 3)
Faculty	Tns1
Submission deadline	12 November 2026, 11:59 PM
Database / file	auvryon_db / auvryon_db.sql (schema + data)
Tested on	MySQL 8.0.46 (Ubuntu); schema also verified earlier on MariaDB 10.11 (XAMPP's engine)

Group members

Name	Student ID
Khandaker Md.Tafsir Hossain	1921945042
Ashraful Kabir Uchas	2413707042
Tahsin Rafid	2422417042
Adnan Shafi	2422272642

1. What was implemented

The tables, constraints and sample data below are the actual MySQL database **auvryon_db**. Every number in this report was measured from the running database with the queries named in the text; nothing is estimated.

Item	Value (measured)
Tables	51
Columns	369
Primary keys	51 (one per table; two link tables use a composite key)
Foreign keys	86
UNIQUE constraints	37
CHECK constraints	8
NOT NULL columns	329 of 369
ENUM columns (fixed state sets)	39
Columns with DEFAULT values	104
AUTO_INCREMENT keys	45
DECIMAL money columns	31
Indexes (incl. PK, FK, UNIQUE)	154
Storage engine	InnoDB (transactions, row locks and enforced foreign keys)
Sample rows inserted	1008 rows; minimum 3 rows in every one of the 51 tables

Project scope in one sentence (from the approved proposal): a responsive PHP/MySQL web application that lets users send, request, split, escrow and track simulated money through a double-entry ledger with built-in protection against wrong transfers. All money in the data is simulated.

2. How to load the database

One file does everything: auvryon_db.sql (PART 1 creates all 51 tables with their constraints, PART 2 inserts the sample data). It drops and re-creates the database auvryon_db, so it can be imported again at any time.

Tool	Steps
MySQL Workbench	File > Open SQL Script > auvryon_db.sql > press the lightning-bolt (Execute). Refresh the SCHEMAS panel: auvryon_db appears with 51 tables.
MySQL command line	mysql -u root -p < auvryon_db.sql (then: mysql -u root -p -e "USE auvryon_db; SHOW TABLES;")
phpMyAdmin (XAMPP)	Start Apache and MySQL in XAMPP, open http://localhost/phpmyadmin, Import tab, choose auvryon_db.sql, Go.
Separate parts	schema.sql (structure only) and sample_data.sql (INSERT statements only) are in the same folder if you want to show them separately.

Quick check after importing

```
USE auvryon_db;
SELECT COUNT(*) AS tables_created FROM information_schema.tables WHERE table_schema='auvryon_db';   -- 51
SELECT COUNT(*) FROM users;  SELECT COUNT(*) FROM transactions;  SELECT COUNT(*) FROM ledger_entries;
```

The sample data was loaded through the application's own services (not typed by hand), which is why balances, ledger entries and statuses are consistent. Demo accounts: tahsin@auvryon.test and others, password Demo@1234, PIN 1234.

3. Consistency with the submitted ERD and Schema

The implementation was compared mechanically with the PDF Auvryon_ERD_and_Schema_Design.pdf that was already submitted: the logical-schema section of that PDF was parsed and compared with the live database (information_schema).

Comparison	In database	Found identical in the ERD document	Result
Relations (tables)	51	51	identical
Columns (name and order, per table)	369	369	identical
Primary-key attributes	53	53	identical
Foreign keys (column and referenced table.column)	86	86	identical
Foreign keys incl. ON DELETE action, nullability and cardinality	86	86	identical

No deviation from the approved design. The 47 entities, 9 weak entities and 4 many-to-many link relations of the ERD are exactly the 51 tables below. Nothing needs to be discussed with the instructor beforehand.

4. Table inventory (51 tables)

#	Table	Purpose	Cols	FKs	Rows
1	**api_tokens**	Hashed bearer tokens for the JSON API.	6	1	3
2	**approval_policies**	Threshold and required approvals per wallet.	5	2	3
3	**approval_policy_approvers**	Designated approvers.	2	2	5
4	**approval_requests**	Payments waiting for approval.	10	4	5
5	**approval_votes**	One vote per approver per request.	5	2	6
6	**audit_logs**	Append-only record of sensitive actions with IP.	8	0	108
7	**budgets**	Personal and shared monthly budgets (group_id 0 = personal).	6	1	7
8	**checkout_sessions**	Expiring QR checkout sessions.	9	2	9
9	**dependent_restrictions**	Blocked categories or merchants.	4	1	3
10	**dependents**	Guardian-dependent link with spending limits.	6	2	3
11	**devices**	Known devices/sessions per user; trust and revoke.	9	1	8
12	**dispute_evidence**	Evidence notes.	5	2	5
13	**disputes**	Dispute cases and admin decisions.	12	4	4
14	**escrow_milestones**	Milestone amounts and funding/closing transactions.	9	3	6
15	**escrows**	Escrow contracts between buyer and seller.	8	2	3
16	**exchange_rates**	SIMULATED dated demonstration FX rates.	6	0	3
17	**expense_groups**	Shared-expense groups.	5	1	3
18	**expense_splits**	Per-member share of an expense.	5	2	26
19	**group_expenses**	Expenses paid by a member.	9	2	8
20	**group_members**	Group membership.	3	2	9
21	**group_settlements**	Settlements linked to real wallet transfers.	7	4	3
22	**idempotency_keys**	Unique (user,key,action) guard against duplicate money requests.	6	1	43
23	**invoice_items**	Invoice line items.	5	1	7
24	**invoice_payments**	Payments reconciled to invoices.	6	3	3
25	**invoices**	Invoices with shareable link token.	11	2	5
26	**iou_repayments**	Partial repayments linked to transactions.	5	2	4
27	**ious**	Loans between users with due dates.	8	2	5
28	**ledger_accounts**	Double-entry accounts (wallet, pocket, hold, escrow, fee, mint, fx, external) with cached balance.	6	0	39
29	**ledger_entries**	Immutable debit/credit lines; the source of truth for balances.	8	2	204
30	**merchants**	Merchant profile for a user.	5	1	3

#	Table	Purpose	Cols	FKs	Rows
31	**money_rules**	Programmable rules (scheduled, allocation, round-up, alert).	16	3	7
32	**notifications**	In-app notifications by category.	7	1	169
33	**offline_queue**	Merchant offline queue awaiting sync.	8	2	6
34	**offline_tokens**	One-time expiring offline payment tokens.	9	3	6
35	**payment_requests**	Money requests between users.	9	3	4
36	**pockets**	Simulated goal pockets backed by pocket ledger accounts.	8	2	5
37	**rate_limits**	Fixed-window counters for login, PIN, transfer and API limits.	3	0	4
38	**recurring_items**	Bills, subscriptions, rent and known income.	12	1	10
39	**reminder_log**	Ensures each reminder is sent once.	6	1	6
40	**risk_alerts**	Rule-based warnings (new recipient, velocity, ghost charge, replay...).	8	1	12
41	**roles**	Lookup of user roles (user, merchant, admin).	2	0	3
42	**rule_runs**	Idempotent run log: unique (rule, run_key).	7	1	12
43	**settings**	Configurable fees, limits, cancellation window.	3	0	15
44	**timeline_events**	Status timelines for escrow, disputes, invoices, transfers, approvals.	7	0	65
45	**token_charges**	Approved/declined merchant charges on tokens.	7	2	10
46	**transactions**	Business-level transaction record and state.	17	4	74
47	**transfer_holds**	Cancellation-window state for held transfers.	6	1	9
48	**user_settings**	Per-user language, privacy and notification preferences.	7	1	11
49	**users**	Accounts: credentials (hashed), role, status, lockout state.	11	1	11
50	**virtual_tokens**	Merchant-specific tokens (hashed) with limits.	10	3	4
51	**wallets**	One wallet per user and currency, bound to a ledger account.	7	2	22

5. Constraints

5.1 Primary keys

Every table has a primary key: a BIGINT UNSIGNED AUTO_INCREMENT surrogate id (the lookup table roles uses a TINYINT id, settings uses the setting key, rate_limits uses the bucket name). Composite primary keys: **group_members (group_id, user_id)** and **approval_policy_approvers (policy_id, user_id)**.

5.2 Foreign keys (86)

72 foreign keys are ON DELETE RESTRICT, so financial history cannot be deleted by accident. The 14 ON DELETE CASCADE keys are owner-dependent child rows only (user_settings, devices, api_tokens, notifications, group_members, group_expenses, expense_splits, approval_policy_approvers, approval_votes, dependent_restrictions, rule_runs, escrow_milestones, dispute_evidence, invoice_items). 'Null?' tells whether participation is optional.

Child column	References	On delete	Null?
api_tokens.user_id	users.id	CASCADE	no
approval_policies.created_by	users.id	RESTRICT	no
approval_policies.wallet_id	wallets.id	RESTRICT	no
approval_policy_approvers.policy_id	approval_policies.id	CASCADE	no
approval_policy_approvers.user_id	users.id	RESTRICT	no
approval_requests.requested_by	users.id	RESTRICT	no
approval_requests.to_wallet_id	wallets.id	RESTRICT	no
approval_requests.txn_id	transactions.id	RESTRICT	yes
approval_requests.wallet_id	wallets.id	RESTRICT	no
approval_votes.approver_id	users.id	RESTRICT	no
approval_votes.request_id	approval_requests.id	CASCADE	no
budgets.user_id	users.id	RESTRICT	no
checkout_sessions.merchant_id	merchants.id	RESTRICT	no
checkout_sessions.paid_txn_id	transactions.id	RESTRICT	yes
dependent_restrictions.dependent_row_id	dependents.id	CASCADE	no
dependents.dependent_id	users.id	RESTRICT	no
dependents.guardian_id	users.id	RESTRICT	no
devices.user_id	users.id	CASCADE	no
dispute_evidence.dispute_id	disputes.id	CASCADE	no
dispute_evidence.user_id	users.id	RESTRICT	no
disputes.admin_id	users.id	RESTRICT	yes
disputes.milestone_id	escrow_milestones.id	RESTRICT	yes
disputes.raised_by	users.id	RESTRICT	no
disputes.txn_id	transactions.id	RESTRICT	yes
escrow_milestones.closing_txn_id	transactions.id	RESTRICT	yes
escrow_milestones.escrow_id	escrows.id	CASCADE	no
escrow_milestones.funded_txn_id	transactions.id	RESTRICT	yes
escrows.buyer_id	users.id	RESTRICT	no
escrows.seller_id	users.id	RESTRICT	no
expense_groups.created_by	users.id	RESTRICT	no
expense_splits.expense_id	group_expenses.id	CASCADE	no
expense_splits.user_id	users.id	RESTRICT	no
group_expenses.group_id	expense_groups.id	CASCADE	no
group_expenses.paid_by	users.id	RESTRICT	no
group_members.group_id	expense_groups.id	CASCADE	no
group_members.user_id	users.id	RESTRICT	no
group_settlements.from_user	users.id	RESTRICT	no
group_settlements.group_id	expense_groups.id	RESTRICT	no

Child column	References	On delete	Null?
group_settlements.to_user	users.id	RESTRICT	no
group_settlements.txn_id	transactions.id	RESTRICT	no
idempotency_keys.user_id	users.id	RESTRICT	no
invoice_items.invoice_id	invoices.id	CASCADE	no
invoice_payments.invoice_id	invoices.id	RESTRICT	no
invoice_payments.payer_id	users.id	RESTRICT	no
invoice_payments.txn_id	transactions.id	RESTRICT	no
invoices.customer_id	users.id	RESTRICT	yes
invoices.issuer_id	users.id	RESTRICT	no
iou_repayments.iou_id	ious.id	RESTRICT	no
iou_repayments.txn_id	transactions.id	RESTRICT	no
ious.borrower_id	users.id	RESTRICT	no
ious.lender_id	users.id	RESTRICT	no
ledger_entries.account_id	ledger_accounts.id	RESTRICT	no
ledger_entries.txn_id	transactions.id	RESTRICT	no
merchants.user_id	users.id	RESTRICT	no
money_rules.target_pocket_id	pockets.id	RESTRICT	yes
money_rules.target_user_id	users.id	RESTRICT	yes
money_rules.user_id	users.id	RESTRICT	no
notifications.user_id	users.id	CASCADE	no
offline_queue.merchant_user_id	users.id	RESTRICT	no
offline_queue.txn_id	transactions.id	RESTRICT	yes
offline_tokens.payee_user_id	users.id	RESTRICT	no
offline_tokens.txn_id	transactions.id	RESTRICT	yes
offline_tokens.user_id	users.id	RESTRICT	no
payment_requests.payer_id	users.id	RESTRICT	no
payment_requests.requester_id	users.id	RESTRICT	no
payment_requests.txn_id	transactions.id	RESTRICT	yes
pockets.account_id	ledger_accounts.id	RESTRICT	no
pockets.user_id	users.id	RESTRICT	no
recurring_items.user_id	users.id	RESTRICT	no
reminder_log.user_id	users.id	RESTRICT	no
risk_alerts.user_id	users.id	RESTRICT	no
rule_runs.rule_id	money_rules.id	CASCADE	no
token_charges.token_id	virtual_tokens.id	RESTRICT	no
token_charges.txn_id	transactions.id	RESTRICT	yes
transactions.created_by	users.id	RESTRICT	no
transactions.from_wallet_id	wallets.id	RESTRICT	yes
transactions.refund_of	transactions.id	RESTRICT	yes
transactions.to_wallet_id	wallets.id	RESTRICT	yes
transfer_holds.txn_id	transactions.id	RESTRICT	no
user_settings.user_id	users.id	CASCADE	no
users.role_id	roles.id	RESTRICT	no
virtual_tokens.merchant_id	merchants.id	RESTRICT	no
virtual_tokens.user_id	users.id	RESTRICT	no
virtual_tokens.wallet_id	wallets.id	RESTRICT	no
wallets.account_id	ledger_accounts.id	RESTRICT	no
wallets.user_id	users.id	RESTRICT	no

5.3 UNIQUE constraints (37)

Table	Constraint	Columns	Business rule
api_tokens	token_hash	token_hash	
approval_policies	wallet_id	wallet_id	
approval_votes	uq_vote	request_id, approver_id	one vote per approver
budgets	uq_budget	user_id, group_id, category	
checkout_sessions	paid_txn_id	paid_txn_id	
checkout_sessions	ref	ref	
dependents	uq_dep	guardian_id, dependent_id	
devices	uq_device	user_id, device_hash	
disputes	ref	ref	
escrow_milestones	closing_txn_id	closing_txn_id	
escrow_milestones	funded_txn_id	funded_txn_id	
escrow_milestones	uq_ms	escrow_id, seq	
escrows	ref	ref	
exchange_rates	uq_rate	base_currency, quote_currency, as_of	
expense_splits	uq_split	expense_id, user_id	one share per member per expense
group_settlements	txn_id	txn_id	
idempotency_keys	uq_idem	user_id, idem_key, action	a payment request can be stored only once
invoice_payments	txn_id	txn_id	
invoices	link_token	link_token	
invoices	number	number	
iou_repayments	txn_id	txn_id	
ledger_accounts	code	code	
merchants	user_id	user_id	
offline_tokens	token_id	token_id	
offline_tokens	txn_id	txn_id	
pockets	account_id	account_id	
reminder_log	uq_rem	ref_table, ref_id, due_date	
roles	name	name	
rule_runs	uq_run	rule_id, run_key	a rule run executes once
transactions	ref	ref	
transactions	refund_of	refund_of	a transaction can be refunded only once
transfer_holds	txn_id	txn_id	
users	email	email	one account per e-mail
users	phone	phone	
virtual_tokens	token_hash	token_hash	
wallets	account_id	account_id	one wallet per user and currency
wallets	uq_wallet_user_cur	user_id, currency	one wallet per user and currency

5.4 CHECK constraints (8)

Table	Constraint	Rule
escrow_milestones	chk_ms_pos	(`amount` > 0)
group_expenses	chk_ge_pos	(`amount` > 0)
ious	chk_iou_pos	(`amount` > 0)
ledger_accounts	chk_nonneg	((`balance` >= 0) or (`kind` in ('mint','fx')))
ledger_entries	chk_entry_pos	(`amount` > 0)
payment_requests	chk_req_pos	(`amount` > 0)
recurring_items	chk_ri_pos	(`amount` > 0)
transactions	chk_amount_pos	(`amount` > 0)

5.5 Other constraints

- **NOT NULL** on 329 of 369 columns; optional columns are exactly the optional participations of the ERD.
- **ENUM** on 39 columns restricts status/type fields to the defined state machines (for example transactions.status: pending, held, completed, cancelled, failed, refunded, disputed).
- **DEFAULT** values on 104 columns (timestamps, statuses, flags, currency BDT).
- **DECIMAL(18,2)** for all money columns, so there is no floating-point rounding.

6. Constraint demonstration (real MySQL output)

Each statement below tries to break a rule. MySQL 8.0.46 rejected every one, and the row counts of all 51 tables were identical before and after. The same statements, with the expected errors, are in constraint_tests.sql.

Duplicate e-mail rejected by UNIQUE (users.email)

```
INSERT INTO users (name,email,password_hash,pin_hash,role_id) VALUES ('Duplicate','tahsin@auvryon.test','x','x',1);
-->  ERROR 1062 (23000) at line 1: Duplicate entry 'tahsin@auvryon.test' for key 'users.email'
```

Foreign key: requester must exist (payment_requests.requester_id -> users.id)

```
INSERT INTO payment_requests (requester_id,payer_id,amount,currency,note) VALUES (99999,1,10,'BDT','ghost user');
-->  ERROR 1452 (23000) at line 1: Cannot add or update a child row: a foreign key constraint fails
   (`auvryon_db`.`payment_requests`, CONSTRAINT `fk_pr_req` FOREIGN KEY (`requester_id`) REFERENCES `users` (`id`) ON DELETE
   RESTRICT)
```

CHECK: a wallet balance can never be negative (ledger_accounts.chk_nonneg)

```
UPDATE ledger_accounts SET balance = -1 WHERE code = 'W-1-BDT';
-->  ERROR 3819 (HY000) at line 1: Check constraint 'chk_nonneg' is violated.
```

CHECK: a transaction amount must be greater than zero (transactions.chk_amount_pos)

```
INSERT INTO transactions (ref,type,status,amount,currency,created_by) VALUES
   ('CHK000000001','transfer','completed',0,'BDT',1);
-->  ERROR 3819 (HY000) at line 1: Check constraint 'chk_amount_pos' is violated.
```

CHECK: ledger entry amounts must be positive (ledger_entries.chk_entry_pos)

```
INSERT INTO ledger_entries (txn_id,phase,account_id,direction,amount,currency) VALUES (1,'post',1,'D',-5,'BDT');
-->  ERROR 3819 (HY000) at line 1: Check constraint 'chk_entry_pos' is violated.
```

NOT NULL: a user must have a name

```
INSERT INTO users (email,password_hash,pin_hash,role_id) VALUES ('noname@example.com','x','x',1);
-->  ERROR 1364 (HY000) at line 1: Field 'name' doesn't have a default value
```

ENUM: only defined transaction states are accepted

```
UPDATE transactions SET status = 'weird' WHERE id = 1;
-->  ERROR 1265 (01000) at line 1: Data truncated for column 'status' at row 1
```

RESTRICT: a user with wallets and history cannot be deleted

```
DELETE FROM users WHERE id = 1;
-->  ERROR 1451 (23000) at line 1: Cannot delete or update a parent row: a foreign key constraint fails
   (`auvryon_db`.`approval_policies`, CONSTRAINT `fk_ap_u` FOREIGN KEY (`created_by`) REFERENCES `users` (`id`) ON DELETE
   RESTRICT)
```

UNIQUE refund_of: a transaction cannot be refunded twice

```
INSERT INTO transactions (ref,type,status,amount,currency,created_by,refund_of) SELECT
   'DUPREF000001','refund','completed',1,'BDT',1,refund_of FROM transactions WHERE refund_of IS NOT NULL LIMIT 1;
-->  ERROR 1062 (23000) at line 1: Duplicate entry '16' for key 'transactions.refund_of'
```

UNIQUE (user_id, idem_key, action): the same payment request cannot be stored twice

```
INSERT INTO idempotency_keys (user_id,idem_key,action) SELECT user_id,idem_key,action FROM idempotency_keys LIMIT 1;
-->  ERROR 1062 (23000) at line 1: Duplicate entry '1-APR4-transfer' for key 'idempotency_keys.uq_idem'
```

UNIQUE (request_id, approver_id): one vote per approver

```
INSERT INTO approval_votes (request_id,approver_id,decision) SELECT request_id,approver_id,'approve' FROM approval_votes
   LIMIT 1;
-->  ERROR 1062 (23000) at line 1: Duplicate entry '1-3' for key 'approval_votes.uq_vote'
```

Rules that depend on several rows (for example "debits equal credits in every posting") cannot be a single CHECK; they are enforced by the application's ledger engine inside one database transaction and verified by the queries in sections 7 and 8.

7. Sample data

1008 rows across all 51 tables, none empty. The data tells one connected story: eleven demo users (four team members, three merchants, a freelancer, a student, a junior and an administrator) who pay each other, split a trip, lend money, pay invoices, use escrow and trigger the safety features. Rows per table:

Table	Rows	Table	Rows	Table	Rows
ledger_entries	204	group_members	9	pockets	5
notifications	169	transfer_holds	9	disputes	4
audit_logs	108	devices	8	iou_repayments	4
transactions	74	group_expenses	8	payment_requests	4
timeline_events	65	budgets	7	rate_limits	4
idempotency_keys	43	invoice_items	7	virtual_tokens	4
ledger_accounts	39	money_rules	7	api_tokens	3
expense_splits	26	approval_votes	6	approval_policies	3
wallets	22	escrow_milestones	6	dependent_restrictions	3
settings	15	offline_queue	6	dependents	3
risk_alerts	12	offline_tokens	6	escrows	3
rule_runs	12	reminder_log	6	exchange_rates	3
user_settings	11	approval_policy_approvers	5	expense_groups	3
users	11	approval_requests	5	group_settlements	3
recurring_items	10	dispute_evidence	5	invoice_payments	3
token_charges	10	invoices	5	merchants	3
checkout_sessions	9	ious	5	roles	3

Every state machine is represented in the data

Column	Values present (count)
transactions.status	completed (71), held (1), cancelled (1), refunded (1)
transactions.type	19 distinct types
transfer_holds.status	held (1), released (7), cancelled (1)
disputes.status	open (1), under_review (1), resolved (1), rejected (1)
invoices.status	partial (1), sent (1), paid (1), overdue (1), cancelled (1)
escrow_milestones.status	released (2), submitted (1), pending (1), disputed (1), refunded (1)
payment_requests.status	pending (1), paid (1), declined (1), cancelled (1)
approval_requests.status	pending (2), rejected (1), executed (2)
virtual_tokens.status	active (2), cancelled (1), paused (1)
money_rules.status	active (5), failed (1), paused (1)
offline_queue.status	settled (3), rejected (2), queued (1)
ious.status	partial (2), open (1), settled (1), cancelled (1)
checkout_sessions.status	paid (6), open (1), cancelled (1), expired (1)
risk_alerts.alert_type	duplicate_expense (1), ghost_charge_blocked (4), offline_replay (1), high_value (1), unusual_amount (1), new_recipient (2), velocity (2)

Sample rows from key tables (real output; passwords and PIN hashes are never shown)

users

```
+----+----------------------------+----------------------+---------+--------+
| id | name                       | email                | role_id | status |
+----+----------------------------+----------------------+---------+--------+
|  1 | Tahsin Rafid               | tahsin@auvryon.test  |       1 | active |
|  2 | Ashraful Kabir Uchas       | uchas@auvryon.test   |       1 | active |
|  3 | Khandaker Md.Tafsir Hossain | tafsir@auvryon.test  |       1 | active |
|  4 | Adnan Shafi                | adnan@auvryon.test   |       1 | active |
|  5 | Aurum Cafe Owner           | merchant@auvryon.test |       2 | active |
+----+----------------------------+----------------------+---------+--------+
```

wallets

id	user_id	currency	account_id	frozen
1	1	BDT	13	0
2	1	USD	14	0
3	2	BDT	15	0
4	2	USD	16	0

transactions

id	ref	type	status	fw	tw	amount	fee	currency
13	AV453CB4A95189	topup	completed	NULL	4	120.00	0.00	USD
15	AVE7729F237109	transfer	completed	3	1	300.00	1.50	BDT
18	AV7FF49F19B5A2	transfer	completed	5	7	700.00	3.50	BDT
30	AV960E5BD7D7DD	iou_repayment	completed	7	5	1000.00	0.00	BDT
42	AVC33CA1803CC1	merchant_payment	completed	1	9	200.00	2.00	BDT

ledger_entries (one transfer = balanced debit and credit lines)

id	txn_id	phase	account_id	direction	amount
27	14	hold	13	D	1206.00
28	14	hold	3	C	1206.00
39	14	release	3	D	1206.00
40	14	release	15	C	1200.00
41	14	release	7	C	6.00

group_expenses

id	group_id	paid_by	description	split_type	amount
1	1	1	Hotel (2 nights)	equal	12000.00
2	1	2	Seafood dinner	custom	4800.00
3	1	3	Car rental	percent	3000.00
4	2	2	October room rent	equal	18000.00
5	2	1	Internet and electricity	equal	2400.00

escrow_milestones

id	escrow_id	seq	title	amount	status
1	1	1	Wireframes	4000.00	released
2	1	2	Visual design	6000.00	submitted
3	1	3	Launch	5000.00	pending
4	2	1	Three concepts	3000.00	disputed
5	3	1	Draft proofreading	800.00	released
6	3	2	Final proofreading	600.00	refunded

invoices

id	number	customer_name	title	due_date	status
1	INV-BEA0DB35	Tahsin Rafid	Mobile app UI kit	2026-11-08	partial
2	INV-2B923440	Walk-in customer	Catering order #12	2026-11-03	sent
3	INV-3FA050AA	Tahsin Rafid	Textbook bundle	2026-11-06	paid
4	INV-01EDCC40	Ashraful Kabir Uchas	Poster design	2026-10-23	overdue
5	INV-CF9D266E	Adnan Shafi	Event catering (cancelled)	2026-11-13	cancelled

virtual_tokens

id	user_id	merchant_id	token_last4	per_charge_limit	monthly_limit	status
1	1	1	ed02	800.00	2000.00	active
2	1	2	668f	600.00	1500.00	active
3	2	3	26f8	300.00	600.00	cancelled
4	2	1	f081	400.00	1200.00	paused

8. Relationships in action (executed queries)

These are the queries listed in presentation_queries.sql; the output below was produced by running them on the delivered database. They show joins across the foreign keys, aggregation and the accounting checks.

1. Every table and how many sample rows it holds (all 51 tables are filled)

```
SELECT table_name, table_rows FROM information_schema.tables WHERE table_schema='auvryon_db' ORDER BY table_rows DESC LIMIT 8;
```

```
+------------------+------------+
| TABLE_NAME       | TABLE_ROWS |
+------------------+------------+
| ledger_entries   |        204 |
| notifications    |        169 |
| audit_logs       |        108 |
| transactions     |         74 |
| timeline_events  |         65 |
| idempotency_keys |         43 |
| ledger_accounts  |         39 |
| expense_splits   |         26 |
+------------------+------------+
```

2. Relationships in action: who paid whom (transactions -> wallets -> users, twice)

```
SELECT t.ref, t.type, fu.name AS payer, tu.name AS payee, t.amount, t.fee, t.status
FROM transactions t
JOIN wallets fw ON fw.id = t.from_wallet_id JOIN users fu ON fu.id = fw.user_id
JOIN wallets tw ON tw.id = t.to_wallet_id   JOIN users tu ON tu.id = tw.user_id
WHERE t.type IN ('transfer','merchant_payment','settlement') ORDER BY t.id DESC LIMIT 8;
```

```
+----------------+------------------+--------------------------+------------------------------+----------+--------+-----------+
| ref            | type             | payer                    | payee                        | amount   | fee    | status    |
+----------------+------------------+--------------------------+------------------------------+----------+--------+-----------+
| AV163F56B8143C | transfer         | Tahsin Rafid             | Demo Junior (dependent)      |   300.00 |   1.50 | cancelled |
| AV70FC657234F0 | transfer         | Tahsin Rafid             | Rafi Freelancer              |  3500.00 |  17.50 | held      |
| AVF6793B2CB6F2 | transfer         | Tahsin Rafid             | Khandaker Md.Tafsir Hossain | 11000.00 |  55.00 | completed |
| AV2C0BCD2DDCD9 | settlement       | Rafi Freelancer          | Tahsin Rafid                 |  1000.00 |   0.00 | completed |
| AVA2B122654E54 | transfer         | Tahsin Rafid             | Khandaker Md.Tafsir Hossain | 20000.00 | 100.00 | completed |
| AVAA3093554E0C | transfer         | Demo Student (dependent) | Ashraful Kabir Uchas         |  1200.00 |   6.00 | completed |
| AVC33CA1803CC1 | merchant_payment | Tahsin Rafid             | Aurum Cafe Owner             |   200.00 |   2.00 | completed |
| AV623714290333 | merchant_payment | Tahsin Rafid             | Dhaka Books Owner            |   123.00 |   1.23 | completed |
+----------------+------------------+--------------------------+------------------------------+----------+--------+-----------+
```

3. Double-entry ledger for one cancelled transfer: hold phase then cancel phase, debits = credits

```
SELECT t.ref, e.phase, a.code AS ledger_account, e.direction, e.amount
FROM ledger_entries e JOIN ledger_accounts a ON a.id = e.account_id JOIN transactions t ON t.id = e.txn_id
WHERE t.status = 'cancelled' ORDER BY e.id;
```

```
+----------------+--------+----------------+-----------+--------+
| ref            | phase  | ledger_account | direction | amount |
+----------------+--------+----------------+-----------+--------+
| AV163F56B8143C | hold   | W-1-BDT        | D         | 301.50 |
| AV163F56B8143C | hold   | SYS-HOLD-BDT   | C         | 301.50 |
| AV163F56B8143C | cancel | SYS-HOLD-BDT   | D         | 301.50 |
| AV163F56B8143C | cancel | W-1-BDT        | C         | 301.50 |
+----------------+--------+----------------+-----------+--------+
```

4. Trial balance: total debits = total credits in each currency

```
SELECT currency, SUM(CASE direction WHEN 'D' THEN amount END) AS total_debits, SUM(CASE direction WHEN 'C' THEN amount END) AS
   total_credits FROM ledger_entries GROUP BY currency;
```

```
+----------+--------------+---------------+
| currency | total_debits | total_credits |
+----------+--------------+---------------+
| BDT      |    373947.50 |     373947.50 |
| USD      |      1040.00 |       1040.00 |
+----------+--------------+---------------+
```

5. Wallet balance equals the sum of its ledger entries (cached balance vs source of truth)

```
SELECT u.name, w.currency, a.balance AS wallet_balance, SUM(CASE e.direction WHEN 'C' THEN e.amount ELSE -e.amount END) AS
   sum_of_entries
FROM wallets w JOIN users u ON u.id = w.user_id JOIN ledger_accounts a ON a.id = w.account_id JOIN ledger_entries e ON
   e.account_id = a.id
WHERE u.email IN ('tahsin@auvryon.test','uchas@auvryon.test') GROUP BY w.id, u.name, w.currency, a.balance;
```

```
+----------------------+----------+----------------+----------------+
| name                 | currency | wallet_balance | sum_of_entries |
+----------------------+----------+----------------+----------------+
| Tahsin Rafid         | BDT      |       47364.00 |       47364.00 |
| Tahsin Rafid         | USD      |         549.75 |         549.75 |
| Ashraful Kabir Uchas | BDT      |       29482.50 |       29482.50 |
| Ashraful Kabir Uchas | USD      |         100.00 |         100.00 |
+----------------------+----------+----------------+----------------+
```

6. Escrow: contract -> milestones -> status (one-to-many, weak entity)

```
SELECT e.title, b.name AS buyer, s.name AS seller, m.seq, m.title AS milestone, m.amount, m.status
FROM escrows e JOIN users b ON b.id = e.buyer_id JOIN users s ON s.id = e.seller_id JOIN escrow_milestones m ON m.escrow_id = e.id
   ORDER BY e.id, m.seq;
```

```
+--------------------+----------------------+-----------------+-----+--------------------+---------+-----------+
| title              | buyer                | seller          | seq | milestone          | amount  | status    |
+--------------------+----------------------+-----------------+-----+--------------------+---------+-----------+
| Website redesign   | Tahsin Rafid         | Rafi Freelancer |   1 | Wireframes         | 4000.00 | released  |
| Website redesign   | Tahsin Rafid         | Rafi Freelancer |   2 | Visual design      | 6000.00 | submitted |
| Website redesign   | Tahsin Rafid         | Rafi Freelancer |   3 | Launch             | 5000.00 | pending   |
| Logo pack          | Adnan Shafi          | Rafi Freelancer |   1 | Three concepts     | 3000.00 | disputed  |
| Thesis proofreading | Ashraful Kabir Uchas | Rafi Freelancer |   1 | Draft proofreading |  800.00 | released  |
| Thesis proofreading | Ashraful Kabir Uchas | Rafi Freelancer |   2 | Final proofreading |  600.00 | refunded  |
+--------------------+----------------------+-----------------+-----+--------------------+---------+-----------+
```

7. Group finance: members, expenses and split type (many-to-many through group_members / expense_splits)

```
SELECT g.name AS grp, ge.description, ge.split_type, ge.amount, u.name AS member, es.share_amount
FROM expense_groups g JOIN group_expenses ge ON ge.group_id = g.id JOIN expense_splits es ON es.expense_id = ge.id JOIN users u ON
   u.id = es.user_id
WHERE g.name = 'Hostel Rent' ORDER BY ge.id, u.name;
```

```
+-------------+--------------------------+------------+----------+----------------------+--------------+
| grp         | description              | split_type | amount   | member               | share_amount |
+-------------+--------------------------+------------+----------+----------------------+--------------+
| Hostel Rent | October room rent        | equal      | 18000.00 | Adnan Shafi          |      6000.00 |
| Hostel Rent | October room rent        | equal      | 18000.00 | Ashraful Kabir Uchas |      6000.00 |
| Hostel Rent | October room rent        | equal      | 18000.00 | Tahsin Rafid         |      6000.00 |
| Hostel Rent | Internet and electricity | equal      |  2400.00 | Adnan Shafi          |       800.00 |
| Hostel Rent | Internet and electricity | equal      |  2400.00 | Ashraful Kabir Uchas |       800.00 |
| Hostel Rent | Internet and electricity | equal      |  2400.00 | Tahsin Rafid         |       800.00 |
+-------------+--------------------------+------------+----------+----------------------+--------------+
```

8. Aggregation: spending by category for one user

```
SELECT t.category, COUNT(*) AS payments, SUM(t.amount + t.fee) AS total FROM transactions t JOIN wallets w ON w.id =
   t.from_wallet_id JOIN users u ON u.id = w.user_id
WHERE u.email = 'tahsin@auvryon.test' AND t.status IN ('completed','held') AND t.type NOT IN
   ('topup','conversion','pocket_deposit','pocket_withdraw') GROUP BY t.category ORDER BY total DESC;
```

```
+---------------+----------+----------+
| category      | payments | total    |
+---------------+----------+----------+
| education     |        6 | 32965.93 |
| escrow        |        3 | 14040.00 |
| invoices      |        3 |  7575.00 |
| transfer      |        3 |  6080.00 |
| food          |        5 |  2337.20 |
| refund        |        1 |   850.00 |
| entertainment |        1 |   800.00 |
| groups        |        1 |   675.00 |
| shopping      |        1 |   202.00 |
| savings       |        2 |    57.00 |
+---------------+----------+----------+
```

9. Five-minute presentation plan

Everyone must be present, so each member has a short part. Open MySQL Workbench with auvryon_db already imported and presentation_queries.sql and constraint_tests.sql open in tabs.

Time	Who (suggested)	What to say and show
0:00-0:30	Tafsir	One sentence on Auvryon. Show the SCHEMAS panel: 51 tables, 11 groups. Say: "Implemented exactly as our submitted ERD: 47 entities plus 4 link tables for many-to-many relationships; 0 deviations."
0:30-1:45	Tafsir	Walk the key relationships: users -> wallets -> ledger_accounts, transactions -> ledger_entries, escrows -> escrow_milestones. Run **query 2** (who paid whom: 5 tables joined) and **query 6** (escrow). Point out weak entities (milestones, invoice items) and link tables (group_members, expense_splits).
1:45-2:45	Uchas	Sample data: run **row_counts.sql** or query 1: every table has data (1,008 rows). Show variety: SELECT status, COUNT(*) FROM transactions GROUP BY status; disputes in all four states; invoices partial / paid / overdue / cancelled.
2:45-4:00	Tahsin	Constraints: open constraint_tests.sql and run 3-4 statements live (duplicate email, foreign key to a missing user, negative balance, amount = 0). Say what each error shows: UNIQUE, FOREIGN KEY, CHECK. Mention 86 FKs, 37 UNIQUE, 8 CHECK, 329 NOT NULL columns.
4:00-5:00	Adnan	Accounting integrity: run **query 4** (trial balance: debits = credits) and **query 5** (wallet balance = sum of ledger entries). Close: "Every payment is posted as balanced ledger lines inside one transaction, so money can never be created, lost or spent twice." Mention the GitHub link.

Likely questions

- **Why surrogate ids?** Stable keys for foreign keys; natural candidates (email, ref, link_token) are kept as UNIQUE.
- **Why DECIMAL?** Exact money arithmetic; no floating-point rounding.
- **Why ON DELETE RESTRICT?** Financial history must not disappear when a user row is touched.
- **Is it normalised?** Yes, 3NF, with documented trade-offs: ledger_accounts.balance is a cached sum of ledger_entries (reconciled by queries 4-5), and budgets.group_id = 0 means "personal budget".
- **Where is "debits = credits" enforced?** By the application's ledger engine in one DB transaction; the database proves it with the trial-balance query and the eight invariants in the project (php bin/reconcile.php).

10. GitHub submission

The repository package auvryon_github_repo.zip is ready to push. A GitHub account is needed to create the repository, so this step must be done by a team member:

```
1. On github.com: New repository > name e.g. auvryon-cse311l > set it to PUBLIC > create (leave it empty).
2. Unzip auvryon_github_repo.zip, open a terminal in the folder, then run:

   git init
   git add .
   git commit -m "Auvryon database implementation: schema + sample data"
   git branch -M main
   git remote add origin https://github.com/YOUR-TEAM/auvryon-cse311l.git
   git push -u origin main

3. Open the repository link in a private/incognito window to confirm it is public and database/auvryon_db.sql is
   visible.
4. Submit the repository link as instructed.
```

Repository layout: **database/auvryon_db.sql** (the file to review), database/schema.sql, database/sample_data.sql, database/queries/*.sql, docs/ (ERD and schema PDF, this report), app/ (the PHP application that uses the database) and a README with the import instructions.

11. What was verified and what was not

Verified (executed)	Evidence
The delivered SQL file imports with no errors on MySQL 8.0.46	import of auvryon_db.sql, exit status 0, 51 tables
Imported row counts equal the source database for all 51 tables	1008 rows compared table by table
The data inserts correctly even with foreign-key checks ON (parent rows come first)	schema + sample_data.sql loaded with FOREIGN_KEY_CHECKS enabled
All 11 constraint-violation statements are rejected and change no data	section 6

Verified (executed)	Evidence
Ledger invariants hold on the imported data (10 checks)	php bin/reconcile.php: 10 PASS
The application runs on this database: 121 service tests and 27 web tests	tests/run_tests.php and tests/http_test.py on MySQL 8.0.46
Schema equals the submitted ERD and Schema	section 3

Not done / limits	Detail
GitHub push	Not performed here (no access to your account). Steps above.
Import on MariaDB/XAMPP of this exact file	The schema part previously loaded on MariaDB 10.11; this final file was run on MySQL 8.0.46 only. If you use XAMPP, test the import once before the class.
Demo data is fictional	Names, e-mails and money are simulated; the four team members appear as demo users.
The instruction mentions the review date as "November 212th"	Assumed to be 12 November 2026; please confirm.